

Du baust im Grunde ein „Brücken-Frontend“, das die Brücke zwischen der mobilen Welt des CEOs (Chat, Sprache) und der darunterliegenden, mächtigen CLI/Agenten-Infrastruktur auf deinem Server schlägt.

Ein CEO will kein Terminal bedienen – er will unterwegs im Auto eine Sprachnachricht einsprechen und das System im Hintergrund arbeiten lassen.

Die Systemarchitektur: Wie die Brücke funktioniert

Da die Agenten (wie z. B. Claude Code oder deine eigenen Python/LangChain-CLI-Agenten) eine laufende Server- oder Terminalumgebung mit Zugriff auf die Repository-Datenstrukturen brauchen, entkoppeln wir das Frontend vollständig von der Ausführung.

Code-Snippet

graph LR

A[CEO Handy: Web App / PWA] -- Chat / Audio --> B(Backend API: FastAPI)

B -- 1. Audio-zu-Text STT --> C(Whisper API)

B -- 2. SSH / Subprocess / MCP --> D[Dein Server: TRACE AI OS CLI]

D -- Führt Befehle aus & liest Logs --> B

B -- Status-Updates & Ergebnis --> A

Die Komponenten für dein Produkt

1. Das Frontend (Die CEO-Oberfläche)

Du entwickelst eine einfache, hochoptimierte **PWA (Progressive Web App)** mit Next.js (React) oder Vue.

- **Zwei Haupt-Buttons:** Ein klassisches Chat-Eingabefeld und ein großer **Mikrofon-Button** für Audio-Aufnahmen (Sprachnachrichten).
- **Vorteil:** Eine PWA läuft auf jedem Smartphone (iOS & Android) wie eine native App, benötigt keinen App-Store und verbraucht kaum Ressourcen, da die Rechenpower auf deinem Server liegt.

2. Das Backend (Der Übersetzer & Brückenwächter)

Hier nutzt du ein performantes Framework wie **FastAPI** (Python). Das Backend übernimmt drei kritische Aufgaben:

- **Audio-Transkription (STT):** Wenn der CEO eine Sprachnachricht schickt, jagst du die Audiodatei durch eine lokale **Whisper-Instanz** (z. B. *Faster-Whisper* auf deiner Inferenz-Hardware) oder nutzt eine schnelle API. Aus *"Hey, check mal das neue Feature im Backend-Repo"* wird sofort sauberer Text.

- **Der CLI-Connector (Die Brücke):** Das Backend nimmt den Text und füttert ihn in deine CLI-Agenten. Das kann auf zwei Wegen passieren:
 1. **Per Subprocess (Lokal):** Wenn FastAPI auf demselben Server läuft wie dein AI OS, startet es die CLI-Befehle im Hintergrund über Python-Subprocesses (z. B. `subprocess.Popen(["claude", "fix the build error"])`).
 2. **Per SSH / Remote-Session (Entkoppelt):** Das Backend verbindet sich via SSH mit der Maschine, auf der das Projekt liegt, startet dort eine persistente Terminal-Session (z. B. via `tmux` oder über das offizielle *Claude Code Remote/Web-Protokoll*) und liest die Ausgaben in Echtzeit aus.

Code-Beispiel: Wie FastAPI die CLI steuert

Damit du verstehst, wie das Backend die Brücke schlägt, ist hier ein einfaches Beispiel in Python (mit Kommentaren zum Nachvollziehen):

Python

```
import subprocess
```

```
from fastapi import FastAPI, HTTPException
```

```
from pydantic import BaseModel
```

```
app = FastAPI(title="TRACE AI-OS CEO Bridge")
```

```
class AgentTask(BaseModel):
```

```
    # Das ist der transkribierte Text von der Sprachnachricht des CEOs
```

```
    prompt: str
```

```
    project_path: str
```

```
@app.post("/api/v1/execute-agent")
```

```
async def execute_cli_agent(task: AgentTask):
```

```
    try:
```

```
        # Wir führen den CLI-Befehl (z.B. claude "aufgabe") im richtigen Projektordner aus
```

```
        # 'capture_output=True' sorgt dafür, dass wir das Terminal-Ergebnis abfangen
```

```
        process = subprocess.run(
```

```
["claude", task.prompt],
cwd=task.project_path,
capture_output=True,
text=True,
timeout=120 # Schutz vor unendlichen Schleifen im Terminal
)
```

```
# Wenn die CLI einen Fehler wirft, fangen wir ihn ab
```

```
if process.returncode != 0:
```

```
    return {
        "status": "error",
        "terminal_logs": process.stderr,
        "message": "Der Agent hat im CLI abgebrochen."
    }
```

```
# Erfolgreiches Ergebnis zurück an die App senden
```

```
return {
    "status": "success",
    "terminal_logs": process.stdout,
    "message": "Agent hat die Aufgabe im Terminal erfolgreich beendet."
}
```

```
except subprocess.TimeoutExpired:
```

```
    raise HTTPException(status_code=408, detail="Agenten-Timeout im Terminal.")
```

```
except Exception as e:
```

```
    raise HTTPException(status_code=500, detail=str(e))
```

Gibt es so etwas schon? (Deine Marktlücke)

Es gibt erste Ansätze in der Open-Source-Community:

- Das Projekt **CloudCLI (auch bekannt als claudecodeui)** versucht gerade genau das: Eine Web- und Mobile-Oberfläche für Claude Code, Cursor CLI und Gemini-CLI bereitzustellen, damit man die Terminal-Agenten von überall steuern kann.

Deine spezifische Marktlücke:

Die meisten dieser Tools sind von Entwicklern für Entwickler gebaut (sie zeigen immer noch Terminal-Code, erfordern Git-Kenntnisse und bieten keine native Audiosteuerung).

Wenn du ein „**Whitelabel CEO-Dashboard**“ baust, das den Terminal-Kram komplett unter einer eleganten Haube versteckt, die Sprachnachrichten perfekt filtert, dem Chef nur die Endergebnisse (z. B. *"Aufgabe erledigt, PR #12 wurde erstellt"*) meldet und im Hintergrund deine extrem mächtige und günstige Docker/VPS-Struktur nutzt – dann hast du ein verdammt starkes Produkt für den B2B-Markt.